

JavaScript String Methods

Complete Guide — Beginner to Advanced

[Examples](#) • [Outputs](#) • [Interview Tips](#) • [Quick Reference](#)

JavaScript String Methods — Complete Guide

A string is a sequence of characters:

```
let str = "Hello JavaScript";
```

Strings are **immutable**, meaning string methods generally return a **new string** rather than modifying the original string.

1. length

Returns the number of characters.

```
let str = "Hello";  
  
console.log(str.length);
```

Output:

```
5
```

Example:

```
let name = "Debarati";  
  
console.log(name.length); // 8
```

2. charAt()

Returns the character at a particular index.

```
let str = "Hello";  
  
console.log(str.charAt(0));  
console.log(str.charAt(1));  
console.log(str.charAt(4));
```

Output:

```
H  
e  
o
```

If index doesn't exist:

```
console.log(str.charAt(10));
```

Output:

```
""
```

3. charCodeAt()

Returns the Unicode/UTF-16 code of a character.

```
let str = "Hello";  
  
console.log(str.charCodeAt(0));
```

Output:

```
72
```

Because **H** has code **72**.

```
console.log("A".charCodeAt(0)); // 65  
console.log("a".charCodeAt(0)); // 97
```

4. at()

Returns the character at an index.

```
let str = "JavaScript";  
  
console.log(str.at(0)); // J
```

```
console.log(str.at(4)); // S
```

The advantage over `charAt()` is that `at()` supports negative indexes.

```
console.log(str.at(-1));
```

Output:

```
t
```

```
console.log(str.at(-2));
```

Output:

```
p
```

5. `toUpperCase()`

Converts string to uppercase.

```
let str = "hello";  
console.log(str.toUpperCase());
```

Output:

```
HELLO
```

6. `toLowerCase()`

Converts string to lowercase.

```
let str = "HELLO";  
console.log(str.toLowerCase());
```

Output:

```
hello
```

Very common in search/login validation:

```
let username = "DEBARATI";  
if (username.toLowerCase() === "debarati") {  
  console.log("Match");  
}
```

7. `trim()`

Removes whitespace from **both sides**.

```
let str = "  Hello World  ";  
console.log(str.trim());
```

Output:

```
Hello World
```

It does not remove spaces between words.

```
let str = "  Hello  World  ";  
console.log(str.trim());
```

Output:

```
Hello  World
```

8. `trimStart()`

Removes whitespace from the beginning.

```
let str = "  Hello";
```

```
console.log(str.trimStart());
```

Output:

```
Hello
```

9. trimEnd()

Removes whitespace from the end.

```
let str = "Hello  ";  
  
console.log(str.trimEnd());
```

Output:

```
Hello
```

10. includes()

Checks whether a string contains another string.

Returns `true` or `false`.

```
let str = "I am learning JavaScript";  
  
console.log(str.includes("JavaScript"));
```

Output:

```
true
```

```
console.log(str.includes("Python"));
```

Output:

```
false
```

Case-sensitive:

```
console.log("Hello".includes("hello")); // false
```

11. startsWith()

Checks whether a string starts with a particular value.

```
let str = "JavaScript";  
  
console.log(str.startsWith("Java"));
```

Output:

```
true
```

```
console.log(str.startsWith("Script"));
```

Output:

```
false
```

You can also specify a position:

```
let str = "Hello JavaScript";  
  
console.log(str.startsWith("Java", 6));
```

Output:

```
true
```

12. endsWith()

Checks whether a string ends with a particular value.

```
let file = "photo.jpg";

console.log(file.endsWith(".jpg"));
```

Output:

```
true
```

Useful for file extensions:

```
if (file.endsWith(".pdf")) {
  console.log("PDF file");
}
```

13. indexOf()

Returns the first position where a value occurs.

```
let str = "Hello World";

console.log(str.indexOf("World"));
```

Output:

```
6
```

Indexes:

```
 H e l l o   W o r l d
0 1 2 3 4 5 6 7 8 9 10
```

If not found:

```
console.log(str.indexOf("JavaScript"));
```

Output:

```
-1
```

14. lastIndexOf()

Returns the position of the **last occurrence**.

```
let str = "Hello Hello";

console.log(str.indexOf("Hello"));
```

Output:

```
0
```

```
console.log(str.lastIndexOf("Hello"));
```

Output:

```
6
```

15. search()

Searches using a string or regular expression.

```
let str = "Hello JavaScript";

console.log(str.search("JavaScript"));
```

Output:

```
6
```

Using regex:

```
let str = "Hello JavaScript";

console.log(str.search(/JavaScript/));
```

Output:

```
6
```

16. match()

Finds matches using a regular expression.

```
let str = "Hello JavaScript";  
  
console.log(str.match(/JavaScript/));
```

With global search:

```
let str = "cat dog cat bird cat";  
  
console.log(str.match(/cat/g));
```

Output:

```
["cat", "cat", "cat"]
```

17. matchAll()

Returns an iterator containing **all matches and their details**.

```
let str = "cat dog cat";  
  
let result = str.matchAll(/cat/g);  
  
console.log([...result]);
```

Useful when you need information such as index and captured groups.

18. replace()

Replaces the first matching occurrence.

```
let str = "Hello World";  
  
let result = str.replace("World", "JavaScript");  
  
console.log(result);
```

Output:

```
["Hello JavaScript"]
```

Original remains unchanged:

```
console.log(str);
```

Output:

```
["Hello World"]
```

19. replaceAll()

Replaces **all occurrences**.

```
let str = "apple apple apple";  
  
console.log(str.replaceAll("apple", "mango"));
```

Output:

```
["mango mango mango"]
```

Difference:

```
"apple apple".replace("apple", "mango");  
// mango apple  
  
"apple apple".replaceAll("apple", "mango");  
// mango mango
```

20. substring()

Extracts part of a string.

```
let str = "JavaScript";  
  
console.log(str.substring(0, 4));
```

Output:

Java

Important:

```
str.substring(start, end)
```

The `end` index is excluded.

```
let str = "JavaScript";  
  
console.log(str.substring(4, 10));
```

Output:

Script

21. `slice()`

Extracts part of a string.

```
let str = "JavaScript";  
  
console.log(str.slice(0, 4));
```

Output:

Java

Negative indexes work:

```
console.log(str.slice(-6));
```

Output:

Script

Another example:

```
console.log(str.slice(-6, -3));
```

Output:

Scr

`slice()` VS `substring()`

```
let str = "JavaScript";  
  
console.log(str.slice(-6));  
// Script  
  
console.log(str.substring(-6));  
// JavaScript
```

`slice()` supports negative indexes; `substring()` does not.

22. `substr()` ⚠

Older/deprecated method.

```
let str = "JavaScript";  
  
console.log(str.substr(0, 4));
```

Output:

Java

Syntax:

```
substr(start, length)
```

Don't use it in new code. Prefer `slice()`.

23. `split()`

Converts a string into an array.

```
let str = "Apple,Banana,Mango";

let result = str.split(",");

console.log(result);
```

Output:

```
["Apple", "Banana", "Mango"]
```

Very important for JavaScript/React/Node.js.

Split by space

```
let str = "Hello World JavaScript";

console.log(str.split(" "));
```

Output:

```
["Hello", "World", "JavaScript"]
```

Split every character

```
let str = "Hello";

console.log(str.split(""));
```

Output:

```
["H", "e", "l", "l", "o"]
```

Limit

```
let str = "A-B-C-D";

console.log(str.split("-", 2));
```

Output:

```
["A", "B"]
```

24. `concat()`

Combines strings.

```
let firstName = "Debarati";
let lastName = "Patwari";

let result = firstName.concat(" ", lastName);

console.log(result);
```

Output:

```
Debarati Patwari
```

But in modern JavaScript, `+` or template literals are usually preferred:

```
let result = `${firstName} ${lastName}`;
```

25. `repeat()`

Repeats a string.

```
let str = "Hello ";

console.log(str.repeat(3));
```

Output:

```
Hello Hello Hello
```


Example:

```
console.log("*".repeat(10));
```

Output:

```
*****
```

26. `padStart()`

Adds characters at the beginning until the string reaches a specified length.

```
let number = "5";  
console.log(number.padStart(3, "0"));
```

Output:

```
005
```

Example:

```
let id = "25";  
console.log(id.padStart(5, "0"));
```

Output:

```
00025
```

Useful for:

```
let hours = "8";  
console.log(hours.padStart(2, "0"));
```

Output:

```
08
```

27. `padEnd()`

Adds characters at the end.

```
let str = "5";  
console.log(str.padEnd(3, "0"));
```

Output:

```
500
```

Example:

```
console.log("Hello".padEnd(10, "."));
```

Output:

```
Hello.....
```

28. `localeCompare()`

Compares two strings according to language/locale sorting rules.

```
console.log("apple".localeCompare("banana"));
```

Usually:

```
-1
```

```
console.log("banana".localeCompare("apple"));
```

Usually:

```
1
```

```
console.log("apple".localeCompare("apple"));
```

Output:

```
0
```

Useful for sorting:

```
let fruits = ["banana", "apple", "mango"];

fruits.sort((a, b) => a.localeCompare(b));

console.log(fruits);
```

29. `normalize()`

Used for Unicode normalization.

This is more advanced and useful when dealing with international text.

```
let str = "é";

console.log(str.normalize());
```

For example, visually identical Unicode strings can sometimes be represented differently internally. `normalize()` helps make them consistent.

Common forms:

```
str.normalize("NFC");
str.normalize("NFD");
str.normalize("NFKC");
str.normalize("NFKD");
```

30. `toString()`

Converts a value to a string.

```
let num = 100;

console.log(num.toString());
```

Output:

```
"100"
```

With a string:

```
let str = "Hello";

console.log(str.toString());
```

Output:

```
Hello
```

31. `valueOf()`

Returns the primitive value of a String object.

```
let str = new String("Hello");

console.log(str.valueOf());
```

Output:

```
Hello
```

This is rarely needed in normal JavaScript.

32. `String.raw()`

A static method of `String`, useful with template literals when you want the raw string.

```
let str = String.raw`Hello\nWorld`;

console.log(str);
```

Output:

```
    Hello\nWorld
```

Notice that `\n` isn't interpreted as a newline.

33. `String.fromCharCode()`

Creates a string from character codes.

```
console.log(String.fromCharCode(72));
```

Output:

```
H
```

Multiple characters:

```
console.log(String.fromCharCode(72, 101, 108, 108, 111));
```

Output:

```
Hello
```

34. `String.fromCodePoint()`

Similar to `fromCharCode()`, but supports the full Unicode code-point range.

```
console.log(String.fromCodePoint(128512));
```

Output:

```
🤖
```

Example:

```
console.log(String.fromCodePoint(65));
```

Output:

```
A
```

35. `codePointAt()`

Returns the Unicode code point at a given position.

```
let str = "🤖";
console.log(str.codePointAt(0));
```

Output:

```
128512
```

This is especially useful with emojis and other characters outside the basic multilingual plane.

36. `isWellFormed()`

Checks whether a string contains valid Unicode surrogate pairs.

```
let str = "Hello 🤖";
console.log(str.isWellFormed());
```

Output:

```
true
```

37. `toWellFormed()`

Returns a string where malformed Unicode surrogate sequences are replaced with the Unicode replacement character.

```
let str = "Hello";
```

```
console.log(str.toWellFormed());
```

Output:

```
Hello
```

This is a more advanced/modern JavaScript method.

Important String Methods for Interviews

You don't need to memorize every method equally.
Focus especially on these:

Method	Main use
<code>length</code>	Get string length
<code>charAt()</code>	Get character
<code>at()</code>	Get character, supports negative index
<code>toUpperCase()</code>	Uppercase
<code>toLowerCase()</code>	Lowercase
<code>trim()</code>	Remove outer spaces
<code>includes()</code>	Check existence
<code>startsWith()</code>	Check beginning
<code>endsWith()</code>	Check ending
<code>indexOf()</code>	Find first position
<code>lastIndexOf()</code>	Find last position
<code>search()</code>	Search with regex
<code>match()</code>	Find regex matches
<code>replace()</code>	Replace first match
<code>replaceAll()</code>	Replace all matches
<code>slice()</code>	Extract part
<code>substring()</code>	Extract part
<code>split()</code>	String → Array
<code>concat()</code>	Join strings
<code>repeat()</code>	Repeat string
<code>padStart()</code>	Add characters at beginning
<code>padEnd()</code>	Add characters at end
<code>localeCompare()</code>	Compare/sort strings

★ Very Important Differences

`slice()` VS `substring()`

```
let str = "JavaScript";

str.slice(-6);    // "Script"
str.substring(-6); // "JavaScript"
```

Remember:

`slice()` → negative indexes supported

`indexOf()` VS `includes()`

```
let str = "JavaScript";

console.log(str.indexOf("Script"));
// 4

console.log(str.includes("Script"));
// true
```

Use `indexOf()` when you need the **position**.

Use `includes()` when you only need **true/false**.

`replace()` VS `replaceAll()`

```
let str = "cat cat cat";

str.replace("cat", "dog");
// dog cat cat

str.replaceAll("cat", "dog");
// dog dog dog
```

`charAt()` VS `at()`

```
let str = "Hello";

str.charAt(0); // H
str.at(0);     // H

str.at(-1);    // o
```

`at()` supports negative indexes.

`split()` VS `join()`

This is very important because `split()` works on strings while `join()` works on arrays.

```
let str = "A,B,C";

let arr = str.split(",");

console.log(arr);
// ["A", "B", "C"]
```

Then:

```
console.log(arr.join("-"));
```

Output:

```
A-B-C
```

So:

```
String → split() → Array → join() → String
```

🔥 Practical Interview Example

Suppose:

```
let email = "  DEBARATI@GMAIL.COM  ";
```

We want to clean and normalize it.

```
let result = email.trim().toLowerCase();

console.log(result);
```

Output:

```
debarati@gmail.com
```

Another example:

```
let sentence = "I am learning JavaScript";

let words = sentence.split(" ");

console.log(words);
```

Output:

```
["I", "am", "learning", "JavaScript"]
```

Then:

```
console.log(words.length);
```

Learn String Methods in This Order

Since you're preparing for **JavaScript/Node.js interviews**, I'd recommend:

Level 1 — Must know

```
length
charAt
at
toLowerCase
toUpperCase
trim
includes
startsWith
endsWith
```

Level 2 — Very important

```
indexOf
lastIndexOf
slice
substring
split
replace
replaceAll
```

Level 3 — Important

```
concat
repeat
padStart
padEnd
search
match
matchAll
localeCompare
```

Level 4 — Advanced

```
normalize
codePointAt
String.fromCharCode
String.fromCodePoint
String.raw
isWellFormed
toWellFormed
```

If you're doing JavaScript interview preparation, `split()`, `slice()`, `substring()`, `replace()`, `replaceAll()`, `indexOf()`, `includes()`, `match()`, and `trim()` **are particularly important**, because they appear frequently in coding questions.